

# Data Quality Errors — Complete Analysis & Fixes

Every error found in `customers_raw.csv` (600 rows), how it was detected, and the SQL that fixes it.

## Summary

| #  | Issue                                  | Level   | Affected Rows | Severity |
|----|----------------------------------------|---------|---------------|----------|
| 1  | <a href="#">Extra Column</a>           | Schema  | 600           | Warning  |
| 2  | <a href="#">Null Violations</a>        | Content | 25            | Critical |
| 3  | <a href="#">Duplicate Primary Keys</a> | Content | 30            | Critical |
| 4  | <a href="#">Invalid Emails</a>         | Content | 20            | Warning  |
| 5  | <a href="#">Invalid Phone Numbers</a>  | Content | 49            | Warning  |
| 6  | <a href="#">Invalid Date Formats</a>   | Content | 38            | Warning  |
| 7  | <a href="#">Out-of-Domain Country</a>  | Content | 28            | Warning  |
| 8  | <a href="#">Out-of-Domain Segment</a>  | Content | 12            | Warning  |
| 9  | <a href="#">Type Drift (is_active)</a> | Content | 30            | Warning  |
| 10 | <a href="#">Whitespace Issues</a>      | Content | 45            | Warning  |

Total: 277+ data quality violations across 10 categories.

## 1. Extra Column ( notes )

Level: Schema

The raw dataset has **10 columns** but the schema defines only **9**. The extra column is `notes`.

```
Schema columns: [customer_id, email, full_name, phone, signup_date, country, city, segment, is_active]
Raw columns:    [customer_id, email, full_name, phone, signup_date, country, city, segment, is_active, notes]

^^^^
```

What `notes` contains:

| Value                      | Count |
|----------------------------|-------|
| <i>(empty)</i>             | 339   |
| VIP                        | 71    |
| imported from legacy CRM   | 69    |
| duplicate of acct C100023? | 64    |

|          |    |
|----------|----|
| verified | 57 |
|----------|----|

**SQL Fix** — Select only schema-defined columns:

```
SELECT
    customer_id, email, full_name, phone,
    signup_date, country, city, segment, is_active
FROM raw_data;
```

**Worth noting:** The `notes` column has some useful metadata — VIP flags, verification status, even someone's manual note about a possible duplicate. In a production pipeline, I'd want to review this before dropping it, or move it to a separate table.

## 2. Null Violations

**Level:** Content

The schema says every column is `nullable: false`, but 6 columns have empty values.

| Column      | Null Count | Example Row IDs           |
|-------------|------------|---------------------------|
| signup_date | 7          | C200410, C200508, C200014 |
| segment     | 6          | C200308, C200440, C200122 |
| full_name   | 4          | C200364, C200508, C200493 |
| phone       | 4          | C200287, C200179, C200069 |
| email       | 2          | C200317, C200429          |
| city        | 2          | C200063, C200553          |

**Why `keep_default_na=False` matters:** By default, pandas converts empty CSV cells to `NaN`. This flag keeps them as empty strings `''`, which makes null detection consistent — we just check `if value.strip() == ''`.

**SQL Fix** — Filter out rows with empty values:

```
SELECT *
FROM raw_data
WHERE TRIM(COALESCE(signup_date, '')) != '';
```

**Note:** Row `C200508` has both `full_name` and `signup_date` empty. Each column is checked independently, so both are caught.

## 3. Duplicate Primary Keys

**Level:** Content

`customer_id` is the primary key, but **15 IDs appear twice** = 30 affected rows.

| Some Duplicate IDs                          |
|---------------------------------------------|
| C200165, C200208, C200238, C200305, C200336 |
| C200371, C200386, C200417, C200444, C200500 |
| C200526, C200565, C200566, C200574, C200583 |

**SQL Fix** — Keep only the first occurrence using `ROW_NUMBER()` :

```
WITH ranked AS (
  SELECT *,
    ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY customer_id) AS _rn
  FROM raw_data
)
SELECT * EXCLUDE (_rn)
FROM ranked
WHERE _rn = 1;
```

**How this works:**

- `PARTITION BY customer_id` — groups rows by ID
- `ROW_NUMBER()` — assigns 1, 2 within each group
- `WHERE _rn = 1` — keeps only the first row
- `EXCLUDE (_rn)` — drops the helper column (DuckDB syntax)

**Trade-off:** We keep the first row, which is arbitrary since there's no timestamp column. Some duplicates have different data in each row (different emails, different phones) — ideally you'd flag those for manual review.

## 4. Invalid Emails

**Level:** Content

20 emails fail the schema's regex `^[^@\s]+@[^\s]+\.[^@\s]+$`. Three sub-types:

**Missing @** (10 rows) — dot instead of @, or truncated:

```
"priya.iyer30.yahoo.com"    — dot where @ should be
"krishna.joshi24"           — completely truncated, no domain at all
```

**Double @@** (5 rows):

```
"neha.mehta89@@yahoo.com"
```

**Trailing whitespace** (4 rows):

```
"sai.malhotra14@yahoo.com "
```

**SQL Fix** — Normalize with lowercase and trim:

```
SELECT * EXCLUDE (email),
      LOWER(TRIM(email)) AS email
FROM raw_data
WHERE NOT regexp_matches(email, '^[^@\\s]+@[^@\\s]+\\.([^@\\s]+)$');
```

**Limitation:** This fixes trailing whitespace and casing, but truncated emails like `krishna.joshi24` can't be fixed programmatically — we don't know the domain. The `@@` ones could be fixed with `REPLACE(email, '@@', '@')` but I kept the SQL simple.

## 5. Invalid Phone Numbers

**Level:** Content

49 phones fail the format `^[+\\d{1,3}-\\d{7,12}]$`. Five variants found:

| Type                     | Count | Example         | Should be      |
|--------------------------|-------|-----------------|----------------|
| Bare 10 digits           | 16    | 7358718098      | +91-7358718098 |
| Spaces instead of hyphen | 13    | +91 85419 06299 | +91-8541906299 |
| Missing + prefix         | 7     | 91-9800667638   | +91-9800667638 |
| Missing hyphen           | 8     | +919352936163   | +91-9352936163 |
| Extra hyphen             | 5     | +91-87037-44630 | +91-8703744630 |

**SQL Fix** — Multi-branch CASE to handle each variant:

```
SELECT * EXCLUDE (phone),
      CASE
        WHEN regexp_matches(phone, '^[+\\d{1,3}-\\d{7,12}]$') THEN phone
        WHEN regexp_matches(phone, '^\\d{1,3}-\\d{7,12}$') THEN '+' || phone
        WHEN regexp_matches(phone, '^\\d{10}$') THEN '+91-' || phone
        WHEN phone LIKE '+%' THEN regexp_replace(
          regexp_replace(phone, '\\s+', '', 'g'),
          '^([+\\d{1,3})(\\d+)$', '\\1-\\2'
        )
        ELSE phone
      END AS phone
FROM raw_data;
```

**Assumption:** For bare 10-digit numbers, I assume `+91` (India) since most of the dataset is Indian customers.

## 6. Invalid Date Formats

**Level:** Content

38 dates don't match the expected `YYYY-MM-DD`. Four different formats found:

| Format               | Count | Example                |
|----------------------|-------|------------------------|
| MM/DD or DD/MM slash | 19    | 03/01/2025, 24/06/2025 |
| YYYY/MM/DD           | 8     | 2024/07/09             |
| DD.MM.YYYY           | 5     | 23.09.2024             |
| DD-Mon-YYYY          | 6     | 12-Mar-2024            |

**SQL Fix** — Uses `TRY_STRPTIME` with `COALESCE` :

```
SELECT * EXCLUDE (signup_date),
       CASE
         WHEN TRIM(signup_date) = '' THEN NULL
         WHEN regexp_matches(signup_date, '^d{4}-d{2}-d{2}$') THEN signup_date
         ELSE strptime(
           COALESCE(
             TRY_STRPTIME(signup_date, '%m/%d/%Y'),
             TRY_STRPTIME(signup_date, '%Y/%m/%d'),
             TRY_STRPTIME(signup_date, '%d/%m/%Y'),
             TRY_STRPTIME(signup_date, '%d.%m.%Y'),
             TRY_STRPTIME(signup_date, '%d-%b-%Y')
           ), '%Y-%m-%d'
         )
       END AS signup_date
FROM raw_data;
```

**Why `TRY_STRPTIME` instead of `strptime` ?**

- `strptime` **crashes** if the format doesn't match
- `TRY_STRPTIME` returns **NULL** on failure — safe to use
- `COALESCE` picks the first format that works

**Date ambiguity:** `03/01/2025` could be March 1 or January 3. I try `MM/DD` first. Dates like `24/06/2025` can only be `DD/MM` (there's no 24th month), so those are unambiguous. For truly ambiguous dates, it's a best-guess — in production you'd want source metadata.

## 7. Out-of-Domain Country Values

**Level:** Content

28 rows have country values outside the allowed list `[IN, US, UK, AE, SG]` .

| Bad Value     | Count | Should be |
|---------------|-------|-----------|
| in            | 9     | IN        |
| United States | 5     | US        |
| INDIA         | 3     | IN        |
| U.K.          | 3     | UK        |

|       |   |    |
|-------|---|----|
| USA   | 3 | US |
| india | 3 | IN |
| India | 2 | IN |

**SQL Fix:**

```
SELECT * EXCLUDE (country),
CASE TRIM(country)
  WHEN 'INDIA' THEN 'IN'
  WHEN 'India' THEN 'IN'
  WHEN 'U.K.' THEN 'UK'
  WHEN 'USA' THEN 'US'
  WHEN 'United States' THEN 'US'
  WHEN 'in' THEN 'IN'
  WHEN 'india' THEN 'IN'
  ELSE TRIM(country)
END AS country
FROM raw_data;
```

I use `TRIM(country)` in both the `CASE` and the `ELSE` branch — that also catches values like `"UK "` with trailing spaces (6 such rows).

## 8. Out-of-Domain Segment Values

**Level:** Content

12 rows have segment values outside `[retail, premium, enterprise]` .

| Bad Value  | Count | Problem      | Should be  |
|------------|-------|--------------|------------|
| Enterprise | 4     | Wrong casing | enterprise |
| Retail     | 3     | Wrong casing | retail     |
| enterprize | 2     | Typo         | enterprise |
| PREMIUM    | 1     | All caps     | premium    |
| Premium    | 1     | Title case   | premium    |
| primium    | 1     | Typo         | premium    |

**SQL Fix:**

```
SELECT * EXCLUDE (segment),
CASE TRIM(segment)
  WHEN 'Enterprise' THEN 'enterprise'
  WHEN 'PREMIUM' THEN 'premium'
  WHEN 'Premium' THEN 'premium'
  WHEN 'Retail' THEN 'retail'
```

```

        WHEN 'enterprize' THEN 'enterprise'
        WHEN 'primium' THEN 'premium'
        ELSE TRIM(segment)
    END AS segment
FROM raw_data;

```

**Why not just LOWER() ?** Because LOWER('enterprize') gives 'enterprize', not 'enterprise'. Typos like enterprize and primium need explicit mapping.

## 9. Type Drift ( is\_active )

Level: Content

is\_active should be "true" or "false", but 30 rows use 9 different boolean formats:

| Bad Value        | Count | Should be          |
|------------------|-------|--------------------|
| 1                | 6     | true               |
| no               | 6     | false              |
| FALSE            | 5     | false              |
| TRUE             | 4     | true               |
| Y                | 4     | true               |
| 0                | 2     | false              |
| yes, True, False | 3     | mapped accordingly |

SQL Fix:

```

SELECT * EXCLUDE (is_active),
       CASE TRIM(is_active)
         WHEN '0' THEN 'false'
         WHEN '1' THEN 'true'
         WHEN 'FALSE' THEN 'false'
         WHEN 'False' THEN 'false'
         WHEN 'TRUE' THEN 'true'
         WHEN 'True' THEN 'true'
         WHEN 'Y' THEN 'true'
         WHEN 'no' THEN 'false'
         WHEN 'yes' THEN 'true'
         ELSE TRIM(is_active)
       END AS is_active
FROM raw_data;

```

## 10. Whitespace Issues

Level: Content

Multiple columns have leading/trailing spaces or double internal spaces.

```
full_name — 23 rows:

" Aadhya Shah"      — leading spaces
"Anaya Verma  "      — trailing spaces
"Aditya  Rao"       — double space between names
```

```
city — 8 rows:

" Hyderabad"        — leading space
"Surat  "            — trailing space
```

```
country / segment — 14 rows:

"UK  "              — trailing space (also caught by domain checker)
"premium  "         — trailing space (also caught by domain checker)
```

SQL Fix (same pattern for all):

```
SELECT * EXCLUDE (full_name),
      TRIM(regexp_replace(full_name, '\s+', ' ', 'g')) AS full_name
FROM raw_data;
```

regexp\_replace('\s+', ' ', 'g') collapses multiple spaces into one, then TRIM() removes leading/trailing.

## SQL Fix Strategies Used

| Strategy          | Used For                             | DuckDB Function                      |
|-------------------|--------------------------------------|--------------------------------------|
| Column selection  | Extra columns                        | SELECT col1, col2, ...               |
| Row filtering     | Null violations                      | WHERE TRIM(col) != ''                |
| Window dedup      | Duplicate keys                       | ROW_NUMBER() OVER (PARTITION BY ...) |
| Regex matching    | Email, phone, date formats           | regexp_matches()                     |
| String cleanup    | Whitespace                           | TRIM(), regexp_replace()             |
| CASE WHEN         | Domain violations, type drift, typos | Explicit value mappings              |
| Safe date parsing | Mixed date formats                   | TRY_STRPTIME() + COALESCE()          |
| String concat     | Missing phone prefix                 | ''+91-                               |